

Test Data Generation Report - sample-qa-test-project

Run QA-sample-qa-test-project-20260805-145451 | Run date: Aug 5, 2026

Executive Summary

The pinned sample QA shop was analyzed and exercised in its explicitly fictional local staging fixture. All nine in-scope page-hosted forms/routes were tested in Chrome; none were skipped under the 25-form cap. The run created 21 relationally valid staging records and confirmed six reproducible validation gaps: 1 Critical, 3 High, 1 Medium, and 1 Low. Six unassigned Jira Bugs were created after project-wide duplicate checks.

The reusable dataset contains 91 field-level and edge-case rows. Reusable valid values are separated from negative probes and manual-review cases. One potential referential-integrity probe was deliberately not submitted because it could create an orphan; it is not counted as a confirmed gap and has no Jira issue.

Dataset:  Test Data Repository - sample-qa-test-project

Repository and Branch/Commit Analyzed

Repository: [sample-qa-test-project](#)

Requested branch: agent/qa-validation-fixture. Resolved and checked out exact commit: 84d392ec8f0d30da10fccdbf00ea2bcfae3922c5. Analysis was limited to that snapshot, including app models, Pydantic schemas, routes/templates, migrations, fixtures/factories/seed, tests, the intentional validation oracle, README, and the staging OpenAPI schema. Vendor and generated dependency content was not treated as application authority.

Staging gate: the repository documents the local /staging and /staging/login paths on 127.0.0.1:8000. The login and dashboard visibly identify STAGING / SAMPLE QA / FICTIONAL DATA ONLY and NOT PRODUCTION. Read-only environment evidence reported environment=staging, production=false, sample_only=true; health reported staging and fictional-test-data-only. The dedicated deliberately fictional qa.staging.runner identity was used, never a personal or production-capable account. The dashboard showed the qa-oracle-v2 clean seed and only the deterministic baseline before writes; the Run Log had no prior run row. No destructive reset was performed.

Forms Discovered and Forms Tested

Inventory was derived from routes and form definitions at the pinned commit. Distinct URLs are counted separately. The dashboard Quick note tab is part of its host page because it does not change the URL or add a navigation level. The hidden /staging/legacy/contact route is explicitly deprecated and excluded. Seven authenticated business/admin form pages plus login and the dashboard host yield nine in-scope forms; all nine were tested.

Form / host	Route	Live result
Staging login	/staging/login	Dedicated fictional login succeeded
Dashboard Quick note host	/staging; POST /staging/forms/flaky-note	Retry-success control; no gap
Registration	/staging/forms/registration	Positive, required-field and malformed-email tests
Customer create	/staging/forms/customer-create; POST /staging/forms/customer	Positive, duplicate email and phone-format tests
Customer edit	/staging/customers/{id}/edit	Phone retest and malformed-email consistency test
Product create	/staging/forms/product	Positive, inert markup and negative-price retests
Order create	/staging/forms/order	Positive, empty-address and over-stock retests
Order item create	/staging/forms/order-item	Positive and negative-quantity retests
QA Admin inventory	/staging/admin; POST /staging/admin/inventory-adjust	No-op and negative-final-stock rejection

Forms discovered: 9. Forms tested: 9. Skipped due to cap: 0. No priority ranking or skipped-form list was needed because the total did not exceed 25.

Dataset Summary

The Test Data Repository tab records module, form, route, field, type/required status, scenario, generated value, expected and actual output, validation type, category, dataset class, pinned rule source, relationship/condition, safety disposition, and generation date. Its categories are happy path, boundary, negative, regression, and UAT. Values use fictional example.com contacts, the sample QA shop domain, draft orders, safe test-kit SKUs, and run-tagged text.

Unique values must be regenerated on a later run and foreign-key IDs resolved from that run's actual state.

Staging record ledger: 1 registration, 2 customers, 5 products, 5 orders, and 8 order items = 21 new records. Customer edit and the delta=0 admin adjustment did not add records. Every parent/child record used an existing FK. OrderItem has no tag text field, so traceability is through the tagged parent order; this schema limitation is disclosed rather than inventing a field.

Validation Rules

- Registration: username 3-40, starts with a letter and uses letters/digits/underscore/hyphen, unique; EmailStr and unique; optional display name <=80; role qa_viewer or qa_editor, default viewer. ORM also checks username length >=3.
- Customer: required name 2-100, unique EmailStr backed by String(254), address_line1 3-120, city 2-80, postal_code 3-20; optional address_line2 <=120 and notes <=1000. Phone is optional but, when nonempty, must match `^\+?[0-9 ()-]{7,24}$`. Status is active/inactive/suspended, default active.
- Product: unique SKU 3-24, normalized uppercase and restricted to uppercase letters, digits and hyphens; name 2-120; optional description <=1000; category office/electronics/home/test-kit. Pydantic price is 0-999999.99 with two decimal places, while Product.validate_unit_price is stricter (>0). Stock is 0-1,000,000 with a nonnegative database check; active defaults true.
- Order: customer_id and product_id must be positive existing references; quantity 1-100 and subject to the stated stock rule; shipping_address 1-200; status draft/pending/paid/shipped/cancelled; shipping method standard/express/pickup; optional gift message <=240. Order number is generated and unique.
- OrderItem: order_id and product_id are required foreign keys, with a unique order/product pair. Quantity is required and the documented application rule says positive and no greater than stock; the database intentionally lacks a quantity check. Unit price is Numeric(10,2), nonnegative by check constraint.
- Admin inventory: resolve an existing product and reject an adjustment whose final stock would be negative. The test used a zero delta for a safe positive path.

Relationship Mapping

Customer 1 -> many Order -> many OrderItem; Product 1 -> many OrderItem.

Order.customer_id references customers.id. OrderItem.order_id references orders.id (cascade on delete in the model) and product_id references products.id. The pair (order_id, product_id) is unique. UserRegistration is a separate non-authentication sample entity. EnvironmentState holds the staging seed marker.

Tagged customer 4 and product 4 formed a draft order and automatic item. A separate item linked tagged order 2 to seeded product 3. Quantity retests used existing tagged parents and product 2; no nonexistent reference was submitted.

Validation Gaps

Every finding below was reconciled against the cited rule, submitted value, observed response and a repeat. Severity follows the requested rubric; the intentional oracle was used as code evidence, not as a substitute for live confirmation.

GAP-04 - Product description reflected as HTML - Critical

Rule: Free text must be HTML-escaped when reflected; the dashboard template applies Jinja |safe to Product.description (app/templates/staging.html; docs/intentional_validation_gaps.json).

Input and procedure: Create a test-kit product with inert `<b data-qa-oracle='GAP-04'>SAMPLE TEST MARKUP</b>` in description, valid unique SKU, price and stock.

Expected: Markup is stored/displayed as literal text, not interpreted.

Actual: The dashboard DOM contained a real `b` element with `data-qa-oracle=GAP-04` and visibly bold SAMPLE TEST MARKUP.

Retest: The identical markup was submitted again under a second unique SKU; two actual elements were observed. Only the SKU changed to satisfy uniqueness.

Severity rationale: Critical because unescaped stored/reflected input opens a security hole. The probe was inert; no script, network call or external action was used.

Jira: [TCW-99](#)

QA Admin: Inventory

OpenAPI / Swagger

Fictional customers

ID	Name	Email	Phone	Status
1	Avery Example	avery.qa.sample@example.com	+1-555-0101	active
2	Jordan Placeholder	jordan.qa.placeholder@example.com	+1-555-0102	active
3	Morgan Sample	morgan.qa.sample@example.com		inactive
4	QA-sample-qa-test-project-20260805-145451 Taylor Example	qa.customer.20260805.145451@example.com	+1-555-0199	active
5	QA-sample-qa-test-project-20260805-145451 Phone Gap	qa.phone.gap.145451@example.com	NOT-A-PHONE	active

Sample products

SKU	Name	Description	Price	Stock
QA-WIDGET-01	Sample QA Widget	Fictional widget for valid-order tests.	19.99	25
DEMO-LAMP-02	Demo Validation Lamp		42.50	4
TEST-PAD-03	Placeholder Test Pad	Not a real product.	7.25	100
QA-145451-VALID	QA-sample-qa-test-project-20260805-145451 Validation Kit	QA-sample-qa-test-project-20260805-145451 fictional order-validation kit	24.95	12
QA-145451-HTML1	QA-sample-qa-test-project-20260805-145451 Markup Probe 1	SAMPLE TEST MARKUP	4.00	4
QA-145451-HTML2	QA-sample-qa-test-project-20260805-145451 Markup Probe 2	SAMPLE TEST MARKUP	4.00	4
QA-145451-PRICE1	QA-sample-qa-test-project-20260805-145451 Price Probe 1	QA-sample-qa-test-project-20260805-145451 negative ORM price probe	-3.25	1
QA-145451-PRICE2	QA-sample-qa-test-project-20260805-145451 Price Probe 2	QA-sample-qa-test-project-20260805-145451 negative ORM price retest	-3.25	1

Live staging evidence: the product table renders SAMPLE TEST MARKUP as bold HTML and displays the separately accepted negative prices.

GAP-01 - Order accepts empty shipping address - High

Rule: OrderCreate.shipping_address has min_length=1 and max_length=200; Order.shipping_address is non-null String(200). The form validates a sentinel but persists the raw input (app/schemas.py, app/models.py, app/main.py).

Input and procedure: Submit a draft order for existing customer ID 4 and product ID 4 with quantity 1 and shipping_address as an empty string.

Expected: Reject before any order or item is saved.

Actual: Success redirect and sample order creation occurred with an empty shipping address.

Retest: The same empty value was resubmitted and accepted again. Existing FKs were retained.

Severity rationale: High because invalid data bypassed a schema/ORM-level required-length rule and was saved, contained to the submitted order.

Jira: [TCW-97](#)

GAP-02 - Customer phone format bypass on create and edit - High

Rule: CustomerCreate.valid_phone requires `^\+?[0-9 ()-]{7,24}$` for a nonempty phone. Both HTML handlers bypass that validator (app/schemas.py and app/main.py).

Input and procedure: Create a tagged customer with NOT-A-PHONE, otherwise valid unique email and address; then edit that same customer and resubmit the same phone.

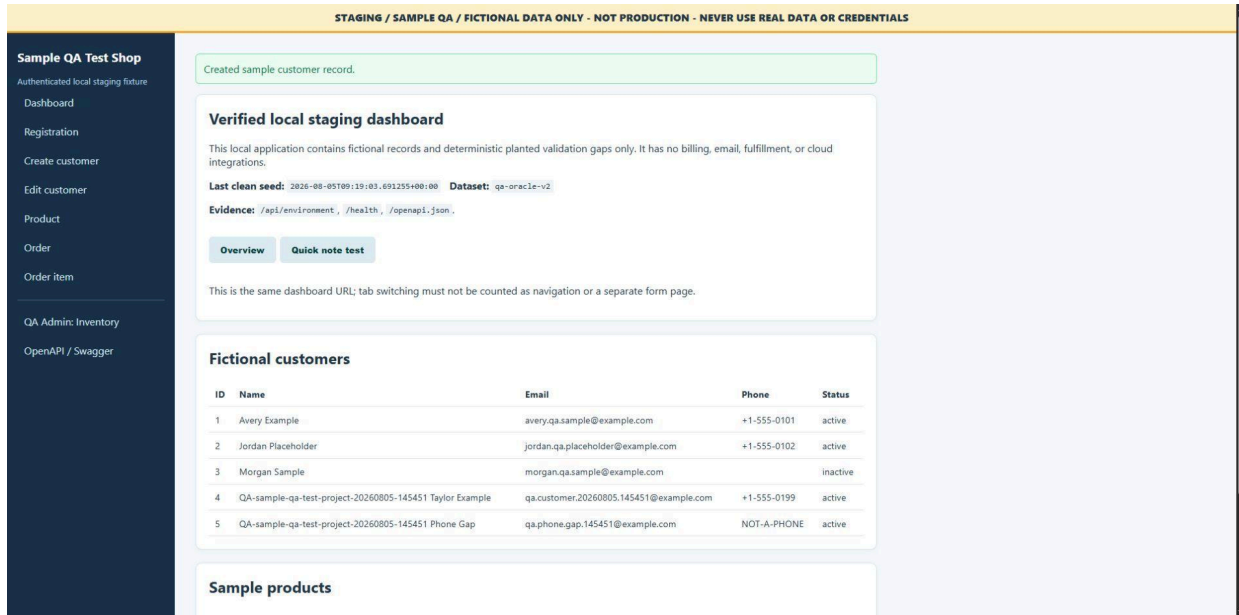
Expected: Reject the defined format on both paths.

Actual: Customer ID 5 was created with NOT-A-PHONE and the dashboard displayed it; the edit path accepted the unchanged violating value.

Retest: Create and edit reproduced the same offending value.

Severity rationale: High because a defined schema-level format rule was bypassed and invalid data persisted in the customer record.

Jira: [TCW-98](#)



Live staging evidence: the tagged customer row displays NOT-A-PHONE beneath the explicit non-production banner.

GAP-06 - Product negative unit price bypasses ORM validator - High

Rule: Product.validate_unit_price is an ORM @validates annotation requiring unit_price > 0; ProductCreate also declares ge=0. The HTML handler uses a Core insert with the raw price (app/models.py, app/schemas.py, app/main.py).

Input and procedure: Create a tagged test-kit product with unit_price -3.25 and otherwise valid fields.

Expected: Reject the negative value.

Actual: The product was saved and the dashboard displayed -3.25.

Retest: The same -3.25 was accepted under a second unique SKU; the SKU alone changed for uniqueness.

Severity rationale: High under the requested rubric because an ORM-level annotation is schema level and invalid data was saved in the submitted product.

Jira: [TCW-101](#)

GAP-05 - Order quantity business rule bypass - Medium

Rule: The documented application rule requires quantity >0 and quantity <= Product.stock_quantity. The OrderItem handler omits both checks and there is no quantity database CHECK (README, docs/intentional_validation_gaps.json, app/main.py, app/models.py).

Input and procedure: Submit OrderItem quantity -7 with existing tagged order and existing product. Separately submit Order quantity 13 against tagged product ID 4 with stock 12.

Expected: Reject before persistence.

Actual: Both negative OrderItem and over-stock Order submissions succeeded. All references existed.

Retest: Quantity -7 succeeded twice on different existing order/product pairs; quantity 13 against stock 12 also succeeded twice. Pair changes only avoided composite uniqueness.

Severity rationale: Medium because the violated positive/stock rule is application-level, not a DB constraint, ORM annotation or migration, and affects submitted records.

Jira: [TCW-100](#)

The Order form manifestation was added to the existing TCW-100 description under "Also seen in" and as a comment; no duplicate issue or severity change was made.

GAP-07 - Inconsistent malformed-email response - Low

Rule: Registration and Customer edit both require a valid email. Their handlers return different message/status behavior for the same underlying failure (app/schemas.py and app/main.py).

Input and procedure: Submit email bad to Registration, then to the tagged Customer edit route.

Expected: Consistent user-facing validation semantics.

Actual: Registration rendered Pydantic technical text with HTTP 200; Customer edit rendered "Enter a valid customer contact email" with HTTP 422. Neither saved an invalid email.

Retest: Both paths were repeated with the same malformed input and the discrepancy held.

Severity rationale: Low because the data was rejected correctly; only the error message/status is inconsistent.

Jira: [TCW-102](#)

Edge Cases Generated

The Sheet separates minimum/maximum lengths, required-empty values, malformed characters, duplicates, allowed-list violations, Unicode and punctuation, oversized text, decimal precision/range, stock boundaries, composite duplicate pairs, and incorrect relationship references. Positive boundary values are marked reusable only when they satisfy every applicable rule. Negative values are never mixed into the reusable class.

Future and expired dates are marked manual review: the scoped forms expose no editable date input; created_at and last_seeded_at are system managed. Nonexistent customer/product/order

references, including the planted GAP-03 product_id=999999 probe, are also manual review and were not submitted because an orphan is explicitly unacceptable. GAP-03 is therefore not a confirmed live finding, is excluded from severity totals, and has no Jira issue.

CONTROL-01 was not counted. The first Quick note submission in this process returned the explicit retry-success response, indicating its stable replay token had already been consumed in process memory. It was not treated as a reproducible validation failure.

Form Validations that have Failed

Form / field	Observed failure	Disposition
Order / shipping_address	Empty required value saved twice	GAP-01 High; TCW-97
Customer create + edit / phone	NOT-A-PHONE saved and retained	GAP-02 High; TCW-98
Product / description	Inert HTML rendered as DOM	GAP-04 Critical; TCW-99
OrderItem / quantity; Order / quantity	-7 and over-stock 13 accepted	GAP-05 Medium; TCW-100
Product / unit_price	-3.25 saved twice	GAP-06 High; TCW-101
Registration vs Customer edit / email	Different error text/status; data rejected	GAP-07 Low; TCW-102

Working controls: browser required-field validation blocked empty Registration inputs; duplicate Customer email was rejected; the admin adjustment that would make stock negative was rejected without mutation. These are not gaps.

Recommendations

1. Remove |safe from product description rendering; escape by default, add output-encoding regression tests using inert markup, and review every display sink.
2. Pass the actual shipping_address and phone through the canonical Pydantic validation path on create and edit. Do not validate a sentinel and then persist raw input.
3. Route product creation through ORM validation or enforce an equivalent positive database CHECK, and test both ORM and Core insertion paths.
4. Enforce positive and stock-bounded quantities in a shared transactional service; add a database positive-quantity CHECK and concurrency-safe stock handling. Cover Order and OrderItem entry points.
5. Enable and verify foreign-key enforcement in the staging database and reject unresolved IDs before insert; validate in an isolated disposable harness before any live orphan probe.
6. Standardize validation error envelopes, messages and HTTP status across HTML entry points. Add contract tests for malformed email and other shared failures.

7. Keep run-tagged, fictional, draft-only data and clean-state evidence. Maintain a non-destructive reset/approval gate before each subsequent run.

Future Datasets Suggestions

- Transactional concurrency and stock race fixtures with isolated rollback, plus quantity-at-stock and stock+1 pairs.
- Unicode normalization, right-to-left, combining-character and escaping cases for unrestricted text while keeping restricted SKU/username cases separate.
- Decimal rounding/scale and extreme Numeric precision cases across HTML, API, ORM and database paths.
- Allowed-list tampering and FK-negative cases in a disposable database that enforces rollback, never by creating live orphans.
- Cross-entry-point error-contract matrices and accessibility/client-validation checks. Add editable date scenarios only if a future form exposes a real date field.

Evidence and Workflow Notes

Jira duplicate review searched the Testing Client Workflows Bug scope and targeted rule terms. The open baseline customer-email uniqueness issue did not cover phone format or message consistency; the repository's closed GAP-02 reference was not reopened. Six new Bugs remain unassigned and carry validation, QA, regression, test-data and bug labels. The Order over-stock manifestation was consolidated into TCW-100.

Chrome evidence is embedded here; attachments were verified on TCW-97 and TCW-99. The extension's file-URL permission blocked further Jira uploads, so the remaining issues retain detailed reproduction steps. This limitation is not a validation finding.